

PROGRAMAÇÃO ORIENTADA A OBJETOS · JAVA

Classes e Objetos

Atributos, métodos e construtores

Prof. Me. Julian Rodrigues Valério

julian.valerio@ifpi.edu.br · IFPI Campus Paulistana · Tecnólogo em ADS

INTRODUÇÃO

Por que Orientação a Objetos?

A POO organiza o programa em **objetos** que representam elementos do mundo real — cada um reunindo seus **dados** e seus **comportamentos**.

Programação estruturada

- Dados e funções ficam separados
- Com o tempo, o código cresce espalhado
- Uma mudança afeta muitos pontos

Orientação a Objetos

- Dados e ações juntos no mesmo objeto
- Código organizado por "coisas" do problema
- Mais fácil reutilizar e manter

ANALOGIA

Pensando como objeto: um celular

O QUE ELE TEM

Características → atributos

- marca
- modelo
- cor
- Recursos de hardware

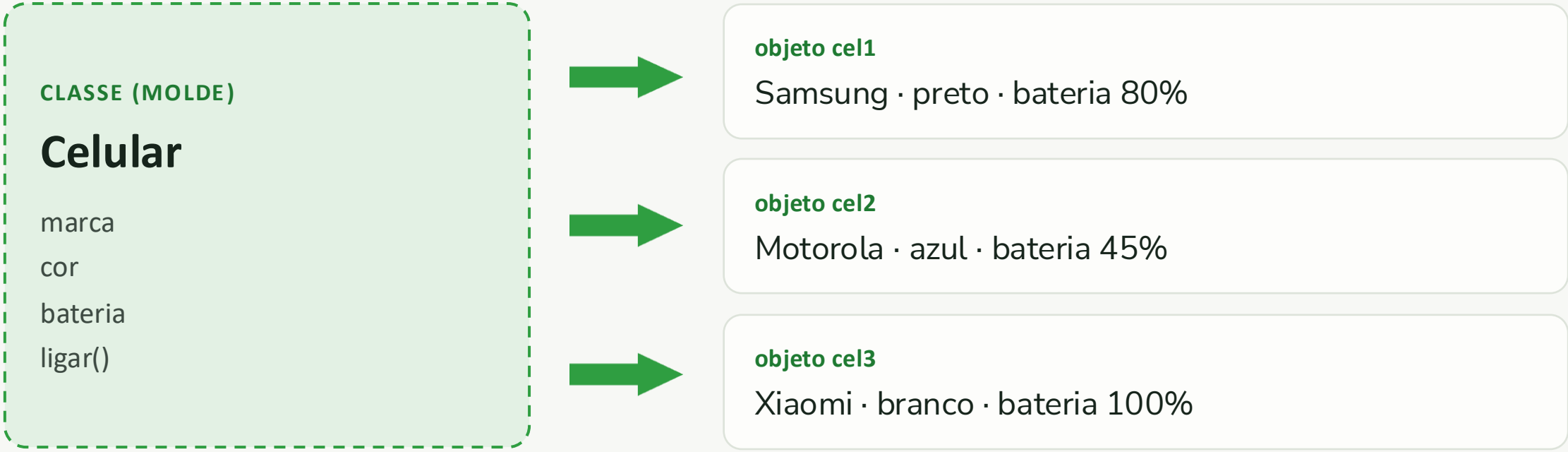
O QUE ELE FAZ

Comportamentos → métodos

- ligar()
- desligar()
- tirarFoto()
- tocarMusica()

CLASSE × OBJETO

Classe é o molde; objeto é o produto



Uma classe → muitos objetos. Cada objeto (ou **instância**) tem seus próprios valores — como a forma de bolo e os bolos.

CLASSE

Anatomia de uma classe em Java

Celular.java

```java
public class Celular {
    // atributos: o que o objeto TEM
    String marca;
    String cor;
    int bateria;

    // métodos: o que o objeto FAZ
    void ligar() {
        System.out.println("Ligando o " + marca);
    }
}
```

1 · Cabeçalho

public class + Nome. O arquivo deve se chamar Celular.java

2 · Atributos

Variáveis da classe: guardam o estado de cada objeto

3 · Métodos

Blocos de código: as ações que o objeto executa

CLASSE

Convenções de nomes em Java

Elemento	Convenção	Exemplo
Classe	PascalCase · substantivo	ContaBancaria
Atributo	camelCase · substantivo	saldoInicial
Método	camelCase · verbo	calcularMedia()
Constante	MAIÚSCULAS com _	TAXA_JUROS

Uma classe public por arquivo, e o arquivo tem o mesmo nome dela: ContaBancaria.java

01

Atributos

As características de cada objeto

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PIAUÍ

ATRIBUTOS

Atributos guardam as características

São **variáveis declaradas dentro da classe** e fora dos métodos.

Cada objeto recebe a **sua própria cópia** de cada atributo.

Formato: **tipo nome;**

Pessoa.java

```
public class Pessoa {
    String nome;      // texto
    int idade;        // inteiro
    double altura;    // real
    boolean estudante; // true/false
}
```

ATRIBUTOS

Atributos já nascem com valor padrão

Tipo do atributo	Valor padrão
byte, short, int, long	0
float, double	0.0
boolean	false
char	'\u0000' (caractere nulo)
String e demais objetos	null (nenhum objeto)

⚠ **Atenção:** variáveis **locais** (dentro de métodos) não têm valor padrão. Usá-las sem inicializar gera erro de compilação.

02

Objetos e referências

Criando instâncias com new e entendendo a memória

OBJETOS

Criando um objeto com new

```
Pessoa p1 = new Pessoa();
```

Pessoa

Tipo da variável: a própria classe

p1

Variável que guarda a referência ao objeto

new

Reserva memória e cria o objeto

Pessoa()

Chama o construtor da classe

Depois de criado, usamos o **ponto (.)** para acessar o objeto: **p1.nome**, **p1.falar()**

OBJETOS

Cada objeto tem seus próprios valores

Principal.java

```java
public class Principal {
    public static void main(String[] args) {
        Pessoa p1 = new Pessoa();
        p1.nome = "Maria";
        p1.idade = 25;

        Pessoa p2 = new Pessoa();
        p2.nome = "João";
        p2.idade = 30;

        System.out.println(p1.nome + " tem " + p1.idade + " anos");
        System.out.println(p2.nome + " tem " + p2.idade + " anos");
    }
}
```

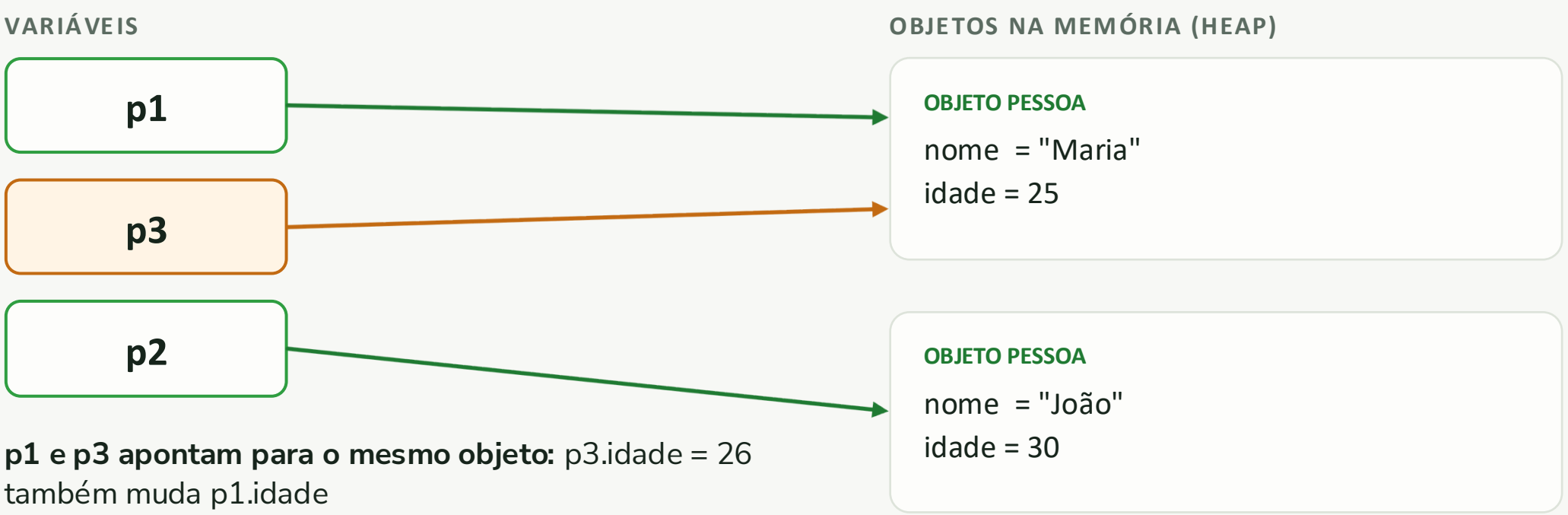
SAÍDA

```
Maria tem 25 anos
João tem 30 anos
```

Mesma classe, **dois objetos independentes**: mudar p1 não altera p2.

A variável guarda uma referência

```
Pessoa p3 = p1;   // copia a referência: NÃO cria um novo objeto
```



p1 e p3 apontam para o mesmo objeto: p3.idade = 26 também muda p1.idade

OBJETOS

null: referência que não aponta para nada

```
Pessoa p4 = null;          // nenhum objeto ainda
System.out.println(p4.nome);  // erro ao executar!
```

SAÍDA (ERRO EM TEMPO DE EXECUÇÃO)

```
Exception in thread "main" java.lang.NullPointerException:
  Cannot read field "nome" ...
```

Crie antes de usar

p4 = new Pessoa();

Ou verifique antes

if (p4 != null) { ... }

03

Métodos

O que os objetos sabem fazer

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PIAUÍ

Estrutura de um método

```
public double calcularMedia(double n1, double n2) { ... }
```

Parte	No exemplo	Função
modificador	public	quem pode usar o método
tipo de retorno	double (ou void)	o que o método devolve
nome	calcularMedia	um verbo, em camelCase
parâmetros	(double n1, double n2)	dados de entrada (opcionais)
corpo	{ ... }	instruções executadas

MÉTODOS

Com ou sem retorno?

SEM RETORNO · VOID

```
void somar(int a, int b) {
   int r = a + b;
   System.out.println("Soma: " + r);
}

// chamada
calc.somar(5, 3);  // imprime Soma: 8
```

COM RETORNO · INT

```
int multiplicar(int a, int b) {
   return a * b;
}

// chamada
int p = calc.multiplicar(4, 2);
System.out.println(p);  // 8
```

return devolve o valor para quem chamou e encerra o método. O valor precisa ser do **tipo declarado** na assinatura.

Métodos que alteram atributos

```
public class Carro {
    String modelo;
    double velocidade;   // começa em 0.0

    void acelerar(double incremento) {
        velocidade += incremento;
    }

    void frear(double decremento) {
        velocidade = Math.max(0, velocidade - decremento);
    }

    void exibirStatus() {
        System.out.printf("%s a %.1f km/h%n", modelo, velocidade);
    }
}
```

+=

Soma e atribui ao próprio atributo

Math.max(0, ...)

Impede velocidade negativa

printf

%s texto · %.1f uma casa decimal · %n nova linha

Chamando os métodos do objeto

```java
public class Principal {
    public static void main(String[] args) {
        Carro c1 = new Carro();
        c1.modelo = "Corolla";

        c1.acelerar(30);
        c1.acelerar(20);
        c1.exibirStatus();

        c1.frear(60);
        c1.exibirStatus();
    }
}
```

SAÍDA

```
Corolla a 50,0 km/h
Corolla a 0,0 km/h
```

Frear 60 com o carro a 50 resulta em **0**, não em −10: o método protege o atributo.

A vírgula em 50,0 vem do idioma do sistema (pt-BR).

MÉTODOS

Parâmetros × argumentos

```
void envelhecer(int anos) {   // anos é o PARÂMETRO
    idade += anos;
}

p1.envelhecer(2);             // 2 é o ARGUMENTO
```

Parâmetro

Variável declarada na assinatura do método. Recebe o valor.

Argumento

Valor concreto enviado no momento da chamada.

Passagem por valor

Java envia sempre uma cópia. Com objetos, copia a referência — e o método altera o objeto original.

MÉTODOS

Sobrecarga de métodos

```
public class Calculadora {
    int somar(int a, int b)          { return a + b; }
    int somar(int a, int b, int c)    { return a + b + c; }
    double somar(double a, double b)  { return a + b; }
}
```

Chamada	Versão usada	Resultado
calc.somar(2, 3)	1ª · dois int	5
calc.somar(2, 3, 4)	2ª · três int	9
calc.somar(2.5, 1.5)	3ª · dois double	4.0

O compilador escolhe pela **lista de parâmetros**. Mudar só o tipo de retorno **não** é sobrecarga — é erro.

04

Construtores

Objetos que já nascem prontos para uso

CONSTRUTORES

O que é um construtor?

Um bloco especial que **inicializa o objeto** no momento em que ele é criado com **new**.

```
public class Pessoa {
    String nome;
    int idade;

    public Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }
}

// uso: um objeto completo em uma linha
Pessoa p = new Pessoa("Maria", 25);
```

- **Mesmo nome** da classe
- **Sem tipo de retorno** — nem void
- Executa **automaticamente** a cada new
- Pode receber **parâmetros**

CONSTRUTORES

O construtor padrão

SEM CONSTRUTOR DECLARADO

```
public class Produto {
    String nome;
}
// o compilador cria sozinho:
// public Produto() { }

Produto p = new Produto();  // ok
```

COM CONSTRUTOR DECLARADO

```
public class Produto {
    String nome;
    public Produto(String nome) {
        this.nome = nome;
    }
}
Produto p = new Produto();  // ERRO!
```

Ao declarar **qualquer** construtor, o padrão deixa de existir. Precisa dos dois? Declare também **public Produto() { }**

CONSTRUTORES

A palavra-chave this

Errado

```
public Pessoa(String nome) {
    nome = nome;       // o parâmetro recebe ele mesmo
}
```

Certo

```
public Pessoa(String nome) {
    this.nome = nome;   // o atributo recebe o parâmetro
}
```

this = referência ao próprio objeto

this.nome → atributo
nome → parâmetro

CONSTRUTORES · ALUNO.JAVA

Exemplo completo: classe Aluno

```java
public class Aluno {
    String nome;
    String matricula;
    double nota;

    public Aluno(String nome, String matricula, double nota) {
        this.nome = nome;
        this.matricula = matricula;
        this.nota = nota;
    }

    boolean estaAprovado() {
        return nota >= 7.0;
    }
}
```

Construtor

Exige os três dados para criar o aluno

matricula: String

Não fazemos contas com ela, e zeros à esquerda são mantidos

estaAprovado()

Retorna true ou false a partir da nota

CONSTRUTORES · PRINCIPAL.JAVA

Criando alunos com o construtor

```
public class Principal {
    public static void main(String[] args) {
        Aluno a1 = new Aluno("Ana", "2025001", 8.5);
        Aluno a2 = new Aluno("Caio", "2025002", 4.0);

        System.out.println(a1.nome + ": " + a1.estaAprovado());
        System.out.println(a2.nome + ": " + a2.estaAprovado());
    }
}
```

SAÍDA

```
Ana: true
Caio: false
```

Teste: escreva new Aluno() sem argumentos. O que o compilador responde? Por quê?

CONSTRUTORES · CONTA.JAVA

Sobrecarga de construtores

```java
public class Conta {
    String titular;
    double saldo;

    // construtor 1
    public Conta(String titular) {
        this.titular = titular;
        this.saldo = 0.0;
    }

    // construtor 2
    public Conta(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }
}
```

O Java escolhe o construtor pelos **argumentos** do new:

> new Conta("Ana")
> → construtor 1 · saldo 0.0

> new Conta("Marcos", 500.0)
> → construtor 2 · saldo 500.0

Percebeu a repetição de this.titular = titular? O próximo slide resolve isso.

CONSTRUTORES

Um construtor chamando outro: this(...)

```
public class Conta {
    String titular;
    double saldo;

    public Conta(String titular) {
        this(titular, 0.0);   // chama o construtor 2
    }

    public Conta(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }
}
```

this(...) chama outro construtor da mesma classe

A inicialização fica em **um só lugar**: menos repetição, menos erros

Regra: até o Java 24, this(...) deve ser a 1ª instrução. No Java 25, pode vir após validações que não usem o objeto.

05

Boas práticas

Encapsulamento e toString()

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PIAUÍ

BOAS PRÁTICAS · CONTABANCARIA.JAVA

Encapsulamento: proteja os atributos

```
public class ContaBancaria {
    private double saldo;        // só a classe acessa

    public double getSaldo() {    // leitura controlada
        return saldo;
    }

    public void depositar(double valor) {
        if (valor > 0) {
            saldo += valor;
        } else {
            System.out.println("Depósito inválido!");
        }
    }
}
```

Modificadores de acesso

private
só a própria classe

(sem modificador)
classes do mesmo pacote

public
qualquer classe

BOAS PRÁTICAS

Usando a classe encapsulada

```
ContaBancaria conta = new ContaBancaria();
conta.depositar(200);
conta.depositar(-50);              // Depósito inválido!
System.out.println(conta.getSaldo()); // 200.0

conta.saldo = 1000;   // ERRO de compilação: saldo é private
```

Convenção	Uso
getNome()	lê o valor de um atributo
setNome(valor)	altera o valor, podendo validar
isAtivo()	leitura de atributo boolean

Nem todo atributo precisa de set: o saldo muda por **depositar()** e **sacar()**, nunca por setSaldo().

BOAS PRÁTICAS

Exibindo objetos com toString()

```java
public class Pessoa {
    private String nome;
    private int idade;
    // construtor omitido

    @Override
    public String toString() {
        return nome + " (" + idade + " anos)";
    }
}
```

SEM TOSTRING · PRINTLN(P1)

Pessoa@1b6d3586

COM TOSTRING · PRINTLN(P1)

Maria (25 anos)

@Override avisa que estamos redefinindo um método herdado. Se o nome estiver errado, o compilador acusa.

REVISÃO

Juntando tudo: o que acontece no new?

```
Pessoa p = new Pessoa("Maria", 25);
```

1

Reserva memória

Espaço para o novo objeto

2

Valores padrão

nome = null
idade = 0

3

Construtor executa

nome = "Maria"
idade = 25

4

Referência

O endereço do objeto vai para p

REVISÃO

Erros comuns de quem está começando

Erro	O que acontece	Como corrigir
Usar objeto sem new	NullPointerException	Criar o objeto antes de usar
nome = nome no construtor	Atributo continua null	Usar this.nome = nome
void no construtor	Vira um método comum	Remover o tipo de retorno
new Pessoa() sem construtor vazio	Erro de compilação	Declarar Pessoa() ou passar os dados
Método int sem return	missing return statement	Retornar em todos os caminhos

EXERCÍCIOS · PARTE 1

Hora de praticar

1 Crie a classe **Livro** com titulo, autor e paginas, e um construtor que receba os três. Crie dois livros e exiba seus dados.

2 Na classe Livro, crie o método **boolean ehLongo()**, que retorna true se o livro tiver mais de 300 páginas.

3 Crie a classe **Retangulo** com base e altura e os métodos calcularArea() e calcularPerimetro(), ambos retornando double.

4 Crie a classe **Lampada** com o atributo boolean ligada e os métodos ligar(), desligar() e exibirEstado().

EXERCÍCIOS · PARTE 2

Desafios

5 Crie a classe **Produto** com nome, preco e quantidade **private**; dois construtores (com e sem quantidade, usando this(...)); e o método valorEmEstoque().

6 Na **ContaBancaria**, acrescente titular, o método sacar(double valor), que recusa saques maiores que o saldo, e um toString() com titular e saldo.

7 **Para pensar:** por que o atributo saldo não deveria ter um setSaldo() público? Responda em até três linhas.

Teste cada classe criando objetos em uma classe **Principal** com o método main.



Dúvidas?

Refaça os exemplos da aula e resolva os exercícios 1 a 7.

Prof. Me. Julian Rodrigues Valério

julian.valerio@ifpi.edu.br · IFPI Campus Paulistana · Tecnólogo em ADS